

OOP Concepts in Java

17 April 2026 05:23

Object oriented programming is a paradigm that provides concepts such as inheritance, data binding, polymorphism, etc.

OOPs(Object Oriented Programmin)

Object Oriented programming aims to implement a real world entities, for example objects, classes, abstraction, inheritance, polymorphism, etc.

Object means a real world entity such as. Pen, chair, table, computer, watch, etc. Object oriented programming is a methodology or paradigm for designing a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Apart from these concepts, there are some other terms which are used in object oriented design:

- Coupling
- Cohesion
- Association
- Aggregation
- Composition

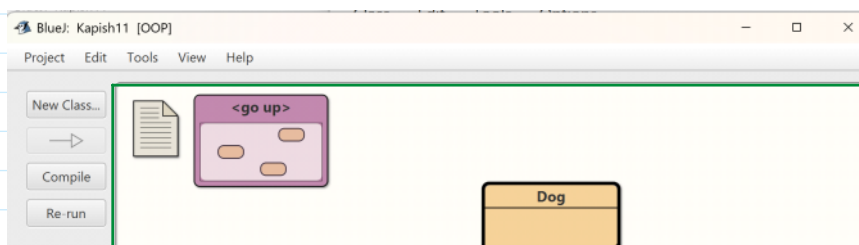
What is an Object?

Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, car, etc. It can be physical or logical.

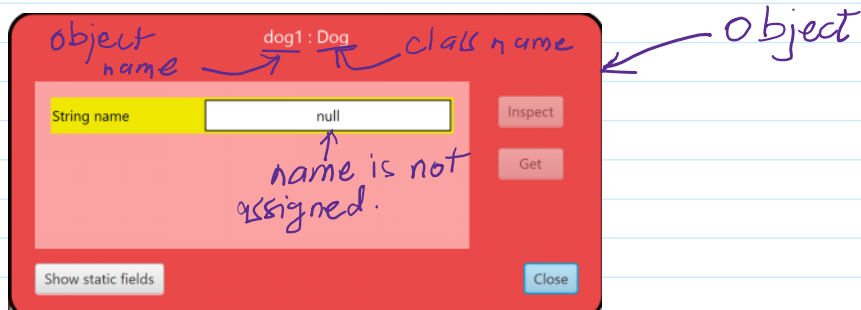
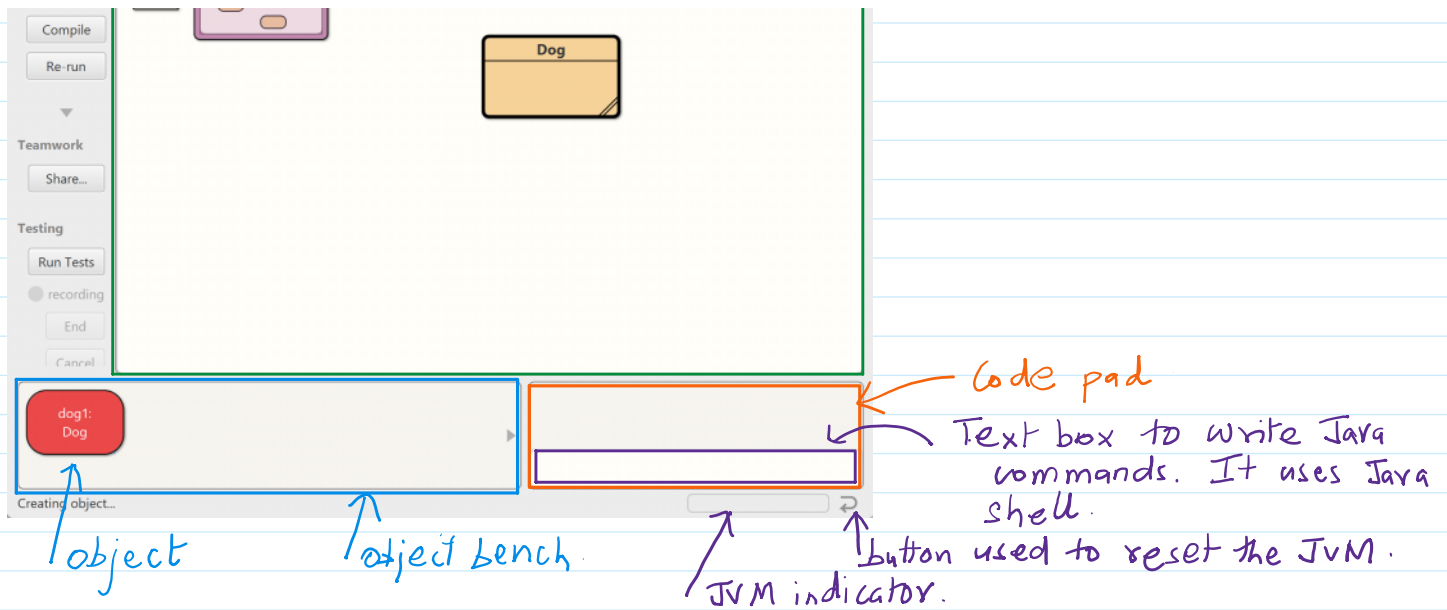
An object can be defined as an instance of a class. It contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the object.

The primary elements of an objects are:

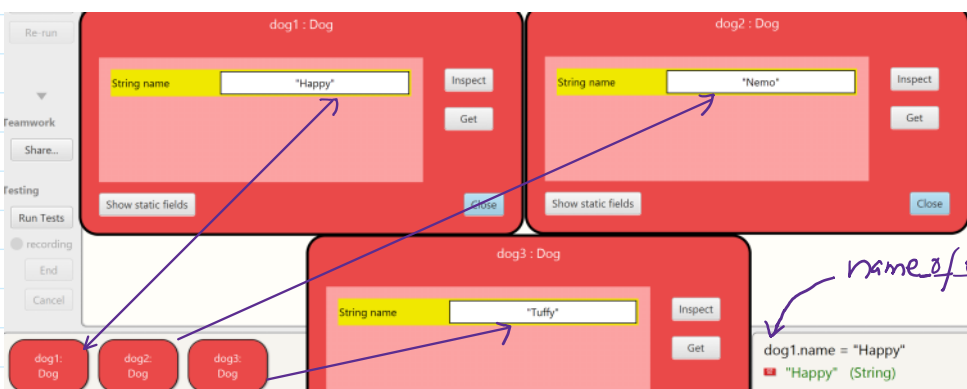
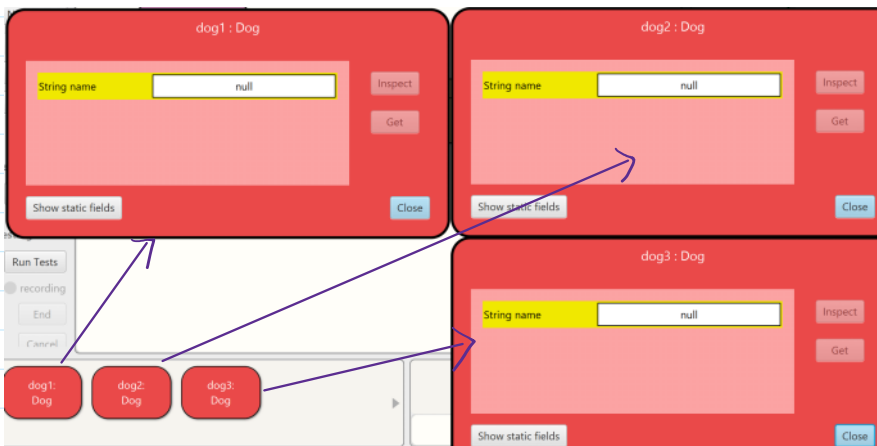
- **State:** It is embodied in an object's properties. Additionally, it reflects an object's attributes.
- **Behavior:** An object's methods are a representation of it. Additionally, it represents how an object reacts to other object.
- **Method:** A method is a group of statements that carry out a certain operation and give an outcome. A method can compete a certain task without giving back any results. Methods are regarded as time saver since they enable us to reuse the code without having to type it again.

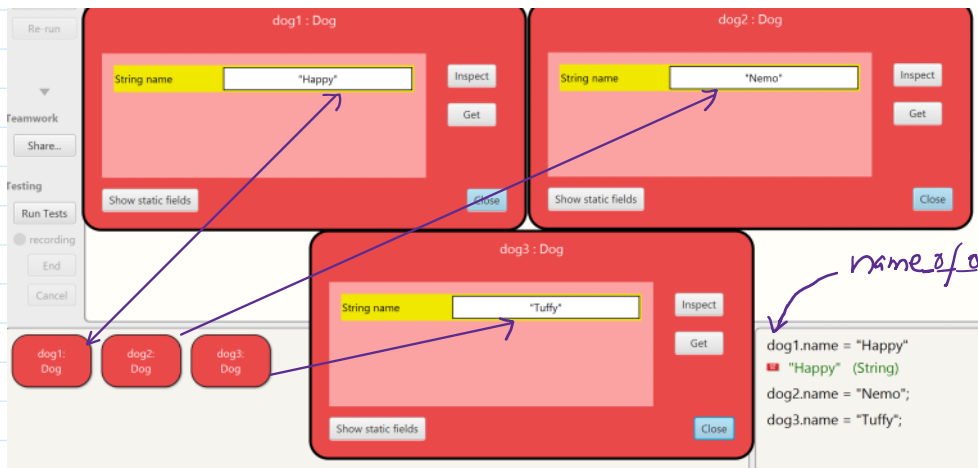


class or package
bench.

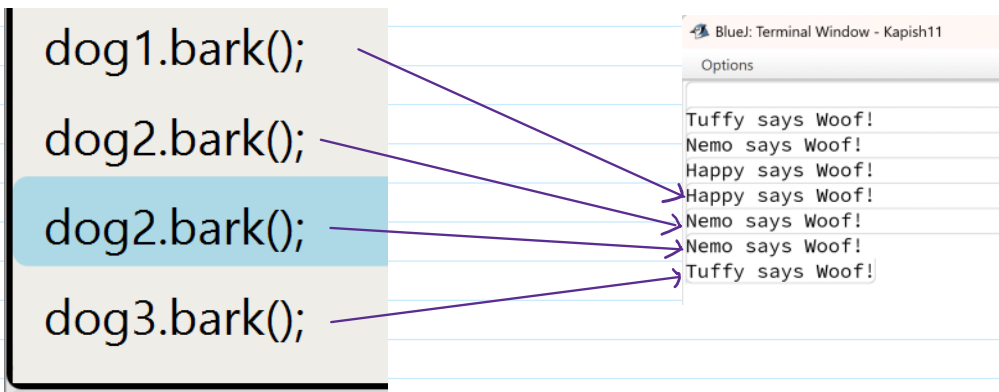
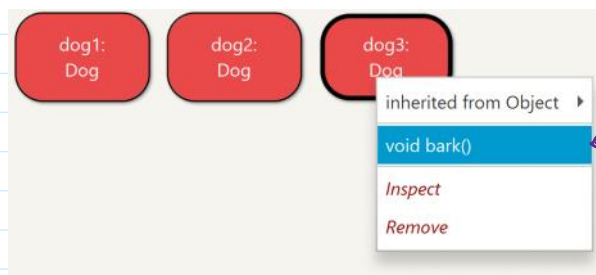


We created three objects of Dog class





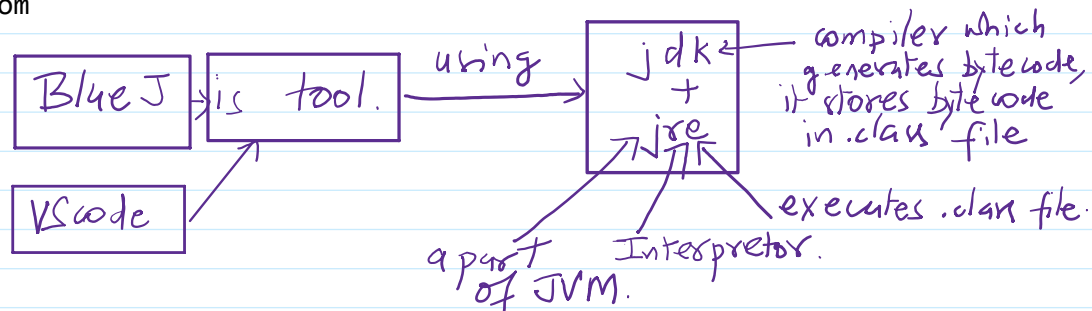
object.name.bark()



What is an IDE?

Integrated Development Environment, it contains code editor or other tools to compile, test and execute the code under one umbrella. For example, Bluej, VSCode, IntelliJ Idea, Eclipse, Netbeans etc.

bluejteacher@gmail.com



Class

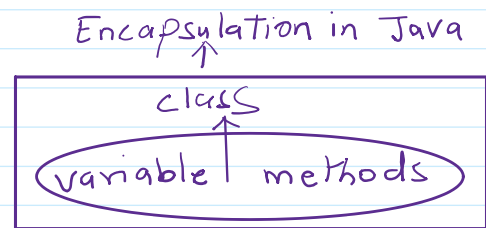
A class is a logical entity. A class can also be defined as a blueprint from which we can create an individual object. Class does not consume any space in the memory.

In general, a class declaration includes the following in the order as it appears:

1. **Modifiers:** A class can be public or have default access.
2. **class keyword:** The class keyword is used to create a class.
3. **Class name:** The name must begin with initial letter (capitalized by convention.)
4. **Body:** The class body surrounds by braces {}.

Encapsulation: Binding or wrapping code and data together into a single unit, known as encapsulation. For example, a capsule is wrapped with different medicines.

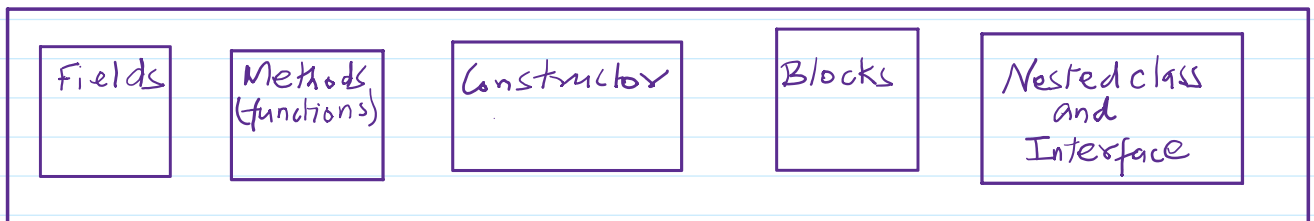
A Java class is an example of encapsulation.
A Java bean is fully encapsulated class, because all data members are private.



What is a class in Java?

A class is a group of objects that have common properties. It is a template or blueprint from which objects are created. We can say that it is an object factory. It is a logical entry. It can't be physical.

A Java class contains.



1. Fields

Variables stated inside a class that indicates the status of an object formed from that class are called fields. Sometimes referred to as instance variables. They specify the data that will be stored in each class object. Different access modifiers such as public, private and protected can be applied to fields to regulate their visibility and usability.

2. Methods

Methods or functions defined inside a class that include the action or behavior that objects of that class are capable of performing. These techniques allow the outside world to function and change the object state (fields) Additionally, methods can be void (that is they return nothing) or

have different access modifiers. They can also return values.

3. Constructors

Constructors are unique methods that are used to initialize class objects. When an object of the class is created using the new keyword, they are called with the same name as the class. Constructors can initialize the fields of an object or carry out any additional setup, that is required when an object is created.

4. Blocks

Within a class, Java allows two different kinds of blocks: Instance block, commonly referred to as initialization blocks and static blocks. Static blocks, which are usually used for static initialization, are only executed once when the class is loaded into memory. Instance block can be used to initialize instance variable and are executed each time a class object is generated.



package OOP;

```
/**
 * Write a description of class Dog here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Dog
{
    String name; //Field or instance variable
    static String str = "has tail";

    {
        //static block
        System.out.print("\nI am a static block");
        System.out.print("\nNotifying you that object is created!");
    }
    public void bark(){ //method or function
        System.out.print("\n" + name + " says Woof!");
        System.out.print("\n" + str);
    }
}
```

```
}
```

5. Nested Class and Interface

Java permits the nesting of classes and interface inside other classes and interfaces. The members (fields, Methods) of the enclosing class are accessible to nested classes, which can be static or non-static. Nested interfaces can be used to logically group related constants and methods together because they are implicitly static.

Syntax:

```
class <class_name>{
    fields;
    methods();
    //nested class
    class <class_name>{
        fields;
        methods();
    }
    ...
}
```